

Universidad Autónoma de Santo Domingo

PRIMADA DE AMÉRICA / Fundada el 28 de octubre de 1538

JOANFI ESPINAL LOPEZ

100519996

Lab Lenguaje de Programación III

INF5170-1

TAREA 3: Realizar un resumen de AJAX.

Radhames Silverio González

Introducción

El desarrollo de aplicaciones web ha evolucionado considerablemente con el objetivo de ofrecer experiencias más rápidas, dinámicas e interactivas a los usuarios. Anteriormente, muchas páginas web necesitaban recargarse completamente cada vez que se solicitaba información nueva o se enviaba un formulario. Este comportamiento podía provocar interrupciones en la navegación y hacer que determinadas operaciones resultaran menos eficientes.

Para mejorar esta experiencia surgió AJAX, una técnica que permite a una página web intercambiar información con un servidor sin tener que recargar todo el documento. Gracias a esta tecnología, es posible actualizar partes específicas de una página, consultar información, validar datos y mostrar resultados mientras el usuario continúa interactuando con la aplicación.

AJAX significa Asynchronous JavaScript and XML. Aunque su nombre hace referencia a XML, actualmente también se utiliza con otros formatos, especialmente JSON. Además, no es un lenguaje de programación independiente, sino una técnica que combina JavaScript, las capacidades de comunicación del navegador y los servicios disponibles en un servidor.

El objetivo de esta investigación es explicar qué es AJAX, cuál es su propósito, cómo funciona y qué relación mantiene con JavaScript, el navegador y el servidor. También se analizarán sus ventajas, desventajas, usos actuales y diferencias con las páginas web tradicionales. Finalmente, se presentará un ejemplo práctico para comprender cómo se utiliza AJAX en una aplicación web.

1. ¿Qué es AJAX?

AJAX es el acrónimo de Asynchronous JavaScript and XML, que en español puede traducirse como JavaScript y XML asíncronos. Se trata de una técnica de desarrollo web que permite enviar y recibir información entre una página web y un servidor sin necesidad de recargar completamente la página.

De acuerdo con MDN Web Docs (s. f.), AJAX describe una técnica mediante la cual las aplicaciones web pueden comunicarse con el servidor y actualizar su contenido de manera dinámica, utilizando tecnologías del navegador como JavaScript y mecanismos de solicitudes HTTP.

AJAX no es un lenguaje de programación, un servidor ni una herramienta única. Es una forma de combinar diferentes tecnologías para permitir que una aplicación web responda a las acciones del usuario de manera más eficiente.

Entre las tecnologías que intervienen en AJAX se encuentran:

- HTML: define la estructura de la página web.
- CSS: controla la apariencia visual de los elementos.
- JavaScript: ejecuta las instrucciones que permiten realizar solicitudes y actualizar el contenido.
- HTTP o HTTPS: facilita la comunicación entre el navegador y el servidor.
- Fetch API o XMLHttpRequest: permiten realizar solicitudes desde JavaScript.
- JSON o XML: pueden utilizarse para representar la información intercambiada.

Actualmente, muchas aplicaciones utilizan la API fetch() de JavaScript para realizar solicitudes asíncronas. Aunque AJAX originalmente hacía referencia a XML, no es obligatorio utilizar este formato. JSON es muy común porque representa datos de una forma compacta y sencilla de procesar.

Por ejemplo, cuando un usuario escribe el nombre de una ciudad en un buscador y aparecen sugerencias sin que la página se recargue, puede estar utilizándose una técnica basada en AJAX.

2. ¿Cuál es el propósito de AJAX?

El propósito principal de AJAX es mejorar la interacción entre el usuario y una aplicación web, permitiendo obtener o enviar información al servidor sin recargar completamente la página.

Esta técnica hace posible que las aplicaciones web respondan a las acciones del usuario de manera más fluida y que actualicen únicamente las secciones que necesitan cambiar.

Sus principales propósitos son los siguientes:

2.1. Actualizar partes de una página web. Permite modificar una sección de la página sin volver a cargar todos sus elementos. Por ejemplo, se puede actualizar una lista de productos sin reconstruir toda la interfaz.

2.2. Enviar y recibir datos de forma asíncrona. El navegador puede realizar una solicitud al servidor mientras el usuario continúa utilizando otras partes de la página, siempre que la aplicación lo permita.

2.3. Mejorar la experiencia del usuario. Reduce las interrupciones que se producen cuando es necesario recargar una página completa para mostrar nuevos resultados.

2.4. Facilitar la comunicación con servicios web. Permite consultar API, enviar formularios, recuperar registros y obtener información actualizada desde un servidor.

2.5. Reducir transferencias innecesarias. En lugar de solicitar una página HTML completa, una aplicación puede solicitar únicamente los datos necesarios para actualizar un componente.

2.6. Crear aplicaciones web dinámicas. Facilita la construcción de interfaces que reaccionan a las acciones del usuario, como paneles de administración, sistemas de búsqueda y plataformas de gestión.

En resumen, AJAX busca que las aplicaciones web sean más interactivas y eficientes. No obstante, su rendimiento depende de factores como la velocidad de la conexión, el tiempo de respuesta del servidor, el volumen de datos y la forma en que se implementa la aplicación.

3. ¿Cómo funciona AJAX en una página web?

AJAX funciona mediante una comunicación entre el navegador y un servidor, coordinada por JavaScript. Cuando el usuario realiza una acción, como presionar un botón o escribir en un campo de búsqueda, JavaScript puede iniciar una solicitud HTTP para obtener o enviar información.

El navegador procesa la respuesta y, si todo funciona correctamente, JavaScript actualiza la parte correspondiente de la página sin necesidad de recargar el documento completo (MDN Web Docs, s. f.-a).

El proceso general se desarrolla en las siguientes etapas:

1. Acción del usuario

El usuario presiona un botón, escribe una búsqueda o envía datos.

2. JavaScript realiza la solicitud

Utiliza Fetch API o XMLHttpRequest para comunicarse con el servidor.

3. El servidor procesa la petición

Ejecuta la operación solicitada y obtiene o prepara los datos.

4. El servidor devuelve una respuesta

Puede enviar datos en formato JSON, XML, texto u otro formato compatible.

5. La página se actualiza

JavaScript interpreta la respuesta y modifica los elementos necesarios.

¿Qué significa que sea asíncrono?

Significa que el código puede iniciar una solicitud y continuar ejecutando otras tareas mientras espera la respuesta. De esta manera, el navegador no necesita detener toda la interacción de la página durante la espera.

La API fetch() utiliza promesas para gestionar las respuestas de las solicitudes. Es importante recordar que una promesa resuelta no significa necesariamente que la respuesta HTTP haya sido exitosa; por eso, el código debe comprobar el estado de la respuesta antes de procesar los datos (MDN Web Docs, s. f.-b).

4. Relación entre AJAX, JavaScript, el navegador y el servidor

AJAX funciona gracias a la colaboración de varios componentes. Cada uno cumple una función específica dentro del proceso de comunicación.

JavaScript: es el encargado de ejecutar las instrucciones que inician las solicitudes, procesan las respuestas y actualizan la interfaz de usuario.

Navegador web: proporciona el entorno donde se ejecuta JavaScript y ofrece herramientas como Fetch API, XMLHttpRequest y el DOM (Document Object Model), que permite modificar el contenido de la página.

Servidor: recibe las solicitudes, procesa la información y devuelve los resultados. Puede estar desarrollado en tecnologías como PHP, Java, Python, Node.js o C#.

Protocolo HTTP o HTTPS: permite intercambiar solicitudes y respuestas entre el navegador y el servidor. HTTPS añade protección mediante cifrado durante la comunicación.

Formato de los datos: representa la información que se envía o recibe. JSON es frecuente en las aplicaciones modernas, aunque también pueden utilizarse XML y texto.

Por ejemplo, en un sistema universitario, JavaScript podría solicitar las asignaturas disponibles al servidor. El servidor consultaría los datos necesarios y devolvería una respuesta. Finalmente, el navegador mostraría las asignaturas en una lista sin recargar toda la página.

La comunicación puede resumirse de esta manera:

- El navegador presenta la página.
- JavaScript inicia la petición.
- El servidor procesa la solicitud.
- La respuesta vuelve al navegador.
- JavaScript actualiza la interfaz.

Es importante señalar que AJAX no reemplaza al servidor ni a la base de datos. Es una técnica que permite que el navegador se comunique con los servicios del sistema de forma asíncrona.

5. Ventajas y desventajas del uso de AJAX

AJAX ofrece beneficios importantes en el desarrollo web, pero también presenta aspectos que deben considerarse al diseñar una aplicación.

5.1. Ventajas

1. Actualización parcial de la página: permite cambiar una sección sin recargar el documento completo.
2. Mayor interactividad: facilita que la aplicación responda rápidamente a las acciones del usuario.
3. Menor transferencia de información en ciertos casos: puede solicitar únicamente los datos necesarios en lugar de descargar una página completa.
4. Mejor experiencia de navegación: evita algunas interrupciones durante las consultas y operaciones.
5. Integración con API: permite consultar servicios externos o internos para obtener información.
6. Validación dinámica de formularios: puede enviar datos al servidor para comprobar si son válidos sin abandonar la página.
7. Compatibilidad con distintas tecnologías: JavaScript puede comunicarse con servidores desarrollados en diferentes lenguajes.

5.2. Desventajas

1. Dependencia de JavaScript: la funcionalidad basada en AJAX necesita que JavaScript esté disponible y funcione correctamente.
2. Manejo de errores: el desarrollador debe gestionar problemas de red, tiempos de espera, respuestas incorrectas y errores del servidor.
3. Complejidad adicional: una aplicación debe controlar los estados de carga, los resultados, los errores y las solicitudes simultáneas.
4. Problemas de seguridad: las solicitudes deben protegerse adecuadamente y el servidor debe validar los datos recibidos.
5. Restricciones entre orígenes: las solicitudes a determinados dominios pueden estar limitadas por las reglas de seguridad del navegador, como CORS.
6. Accesibilidad e historial: si la aplicación no se diseña correctamente, los cambios dinámicos pueden dificultar la navegación con herramientas de asistencia o el uso de los botones de atrás y adelante.
7. Dependencia de la conexión: cuando la información necesita obtenerse del servidor, una conexión lenta o interrumpida puede retrasar los resultados.

En conclusión, AJAX ofrece ventajas importantes cuando se utiliza correctamente. Sin embargo, debe implementarse con un manejo adecuado de errores, controles de seguridad y una interfaz que informe al usuario sobre el estado de las operaciones.

6. Usos actuales de AJAX

AJAX continúa siendo una técnica útil en el desarrollo de aplicaciones web que necesitan obtener información del servidor sin recargar completamente la página. Aunque el término se menciona menos que antes, el concepto sigue presente en muchas aplicaciones que utilizan Fetch API, XMLHttpRequest y bibliotecas modernas de JavaScript (MDN Web Docs, s. f.-c).

Sus principales usos incluyen los siguientes:

6.1. Formularios dinámicos

Permite enviar formularios y mostrar una confirmación o un mensaje de error sin redirigir al usuario a otra página.

Por ejemplo, un formulario de contacto puede enviar el nombre, el correo electrónico y el mensaje al servidor. Cuando se recibe la respuesta, la página muestra si el envío fue exitoso.

6.2. Búsquedas en tiempo real

Se utiliza para consultar información a medida que el usuario escribe. Una aplicación puede enviar el término de búsqueda al servidor y mostrar resultados relacionados sin recargar la página.

Este mecanismo se utiliza en buscadores de productos, catálogos y sistemas de consulta.

6.3. Validaciones de datos

Permite verificar información mediante solicitudes al servidor. Por ejemplo, un sistema puede consultar si un nombre de usuario ya existe antes de completar el registro.

La validación realizada en el navegador mejora la experiencia del usuario, pero el servidor siempre debe volver a validar los datos para garantizar la seguridad.

6.4. Carga de datos

AJAX facilita recuperar listas de usuarios, productos, asignaturas o transacciones y mostrarlas en una tabla. También puede utilizarse para cargar más resultados cuando el usuario se desplaza por una lista.

6.5. Aplicaciones web dinámicas

Los paneles administrativos, los sistemas de gestión y otras aplicaciones pueden utilizar AJAX para actualizar indicadores, notificaciones y registros sin reconstruir toda la página.

6.6. Sistemas empresariales

En un sistema de inventario, AJAX puede utilizarse para consultar la disponibilidad de un producto, actualizar una cantidad o mostrar información relacionada con un pedido.

En un sistema financiero, puede utilizarse para consultar movimientos o actualizar datos de una operación, siempre que la aplicación implemente la autenticación, la autorización y las medidas de seguridad necesarias.

6.7. Consumo de servicios web

Permite consultar API para obtener información, como catálogos, datos públicos o información empresarial. La aplicación recibe la respuesta y utiliza JavaScript para presentarla al usuario.

En todos estos casos, AJAX permite separar la comunicación con el servidor de la actualización visual de la página, lo que contribuye a construir interfaces más dinámicas.

7. Diferencia entre una página web tradicional y una página que utiliza AJAX

Las páginas web tradicionales y las que utilizan AJAX pueden acceder a información del servidor, pero se diferencian en la forma en que presentan los resultados y actualizan el contenido.

En una página tradicional, al enviar un formulario o solicitar una nueva página, normalmente el navegador recibe un nuevo documento HTML y lo carga. En una aplicación que utiliza AJAX, JavaScript puede solicitar los datos necesarios y actualizar una parte específica de la interfaz.

Característica	**Página web tradicional**	**Página web con AJAX**
Actualización del contenido	Generalmente recarga la página completa al navegar o enviar formularios	Puede actualizar solo una sección
Comunicación con el servidor	Puede implicar una navegación completa	Puede realizar solicitudes en segundo plano
Experiencia del usuario	Puede haber interrupciones durante las recargas	Puede ofrecer una interacción más fluida
Datos transferidos	Puede descargar un documento HTML completo	Puede recibir solamente los datos necesarios
Uso de JavaScript	Puede ser limitado o no ser esencial para la navegación básica	Generalmente coordina las solicitudes y actualizaciones
Manejo de errores	Puede mostrar una nueva página con el resultado	Puede mostrar mensajes directamente en la interfaz
Ejemplo	Enviar un formulario y cargar una página de confirmación	Enviar el formulario y mostrar la confirmación en la misma página

Es importante aclarar que AJAX no impide que una página se recargue. Una aplicación puede combinar navegación tradicional con solicitudes AJAX según las necesidades de cada operación.

Por ejemplo, una tienda virtual podría utilizar navegación tradicional para abrir la página de un producto y AJAX para actualizar la cantidad del carrito de compras. Así, el usuario puede seguir explorando los productos sin que cada modificación del carrito requiera recargar todo el documento.

8. Ejemplo sencillo del funcionamiento de AJAX

Para comprender cómo funciona AJAX en la práctica, se desarrollará un ejemplo que consulta una API pública y muestra los datos de un usuario en una página web.

El ejemplo utilizará JavaScript y Fetch API para solicitar información de la API de prueba JSONPlaceholder. Esta API proporciona datos ficticios que se pueden utilizar para practicar solicitudes HTTP.

8.1. Código completo del ejemplo

Crea un archivo llamado index.html, copia el siguiente código y ábrelo en el navegador. Al presionar el botón, se consultarán los datos del usuario número 1.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Ejemplo de AJAX</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f0f4f8;
            padding: 30px;
            text-align: center;
        }

        .contenedor {
            max-width: 500px;
            margin: auto;
            padding: 25px;
```

```
            background: white;
            border-radius: 10px;
        }

        button {
            padding: 12px 20px;
            background: #1769aa;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        #resultado {
            margin-top: 20px;
            text-align: left;
        }
    </style>
</head>

<body>
    <div class="contenedor">
        <h1>Consulta de usuarios</h1>
        <p>Ejemplo práctico de AJAX con JavaScript.</p>

        <button id="consultar">Consultar usuario</button>

        <div id="resultado" aria-live="polite">
            Presiona el botón para consultar los datos.
        </div>
    </div>
```

```
<script>
    const boton = document.getElementById("consultar");
    const resultado = document.getElementById("resultado");

    boton.addEventListener("click", async function () {
        resultado.textContent = "Cargando información...";
        boton.disabled = true;

        try {
            const respuesta = await fetch(
                "https://jsonplaceholder.typicode.com/users/1"
            );

            if (!respuesta.ok) {
                throw new Error("No se pudo obtener la información.");
            }

            const usuario = await respuesta.json();

            const titulo = document.createElement("h2");
            titulo.textContent = usuario.name;

            const correo = document.createElement("p");
            correo.textContent = "Correo: " + usuario.email;

            const ciudad = document.createElement("p");
            ciudad.textContent = "Ciudad: " + usuario.address.city;

            resultado.replaceChildren(titulo, correo, ciudad);
```

```
        } catch (error) {
            resultado.textContent =
                "Ocurrió un error al consultar los datos.";
        } finally {
            boton.disabled = false;
        }
    });
  </script>
</body>
</html>
```

8.2. Explicación del código

1. Estructura HTML

El código contiene un título, un botón y un elemento div con el identificador resultado. Este último se utiliza para mostrar los datos que devuelve el servidor.

2. Estilos CSS

El bloque style define los colores, el tamaño del contenedor, los espacios y la apariencia del botón. Estos estilos mejoran la presentación visual, pero no realizan la comunicación con el servidor.

3. Selección de elementos con JavaScript

Las instrucciones document.getElementById() permiten localizar el botón y el área donde se presentará la información.

4. Evento de clic

La instrucción addEventListener("click", ...) ejecuta una función cuando el usuario presiona el botón. En ese momento, el programa muestra un mensaje de carga y desactiva temporalmente el botón.

5. Solicitud AJAX mediante Fetch API

La instrucción fetch() envía una solicitud HTTP a la dirección de la API. La palabra clave await permite esperar el resultado de forma asíncrona, sin bloquear toda la interfaz mientras llega la respuesta.

6. Procesamiento de la respuesta

El programa comprueba respuesta.ok para detectar errores HTTP y luego utiliza respuesta.json() para convertir los datos recibidos en un objeto de JavaScript.

7. Actualización de la página

Las instrucciones createElement(), textContent y replaceChildren() crean y muestran los datos del usuario en el área de resultados. No es necesario recargar la página completa.

8. Manejo de errores

El bloque catch muestra un mensaje si ocurre un problema durante la solicitud o el procesamiento. El bloque finally vuelve a habilitar el botón, independientemente de que la operación haya terminado correctamente o haya fallado.

Este ejemplo sigue el funcionamiento habitual de Fetch API: realizar una solicitud, comprobar la respuesta, procesar los datos y actualizar la interfaz (MDN Web Docs, s. f.-b).

8.3. ¿Cómo comprobar que funciona?

Paso 1. Crear el archivo

Guarda el código como index.html en una carpeta de tu computadora.

Paso 2. Abrir la página

Abre el archivo en un navegador con conexión a Internet.

Paso 3. Presionar el botón

Haz clic en “Consultar usuario”. Aparecerá un mensaje de carga mientras se obtiene la respuesta.

Paso 4. Observar el resultado

Si la solicitud tiene éxito, se mostrarán el nombre, el correo electrónico y la ciudad del usuario devuelto por la API.

El ejemplo requiere conexión a Internet para acceder a la API. Si no funciona, comprueba la conexión y la disponibilidad del servicio.

8.4. Importancia del ejemplo

Este ejercicio demuestra que AJAX permite obtener información de un servidor y mostrarla en una página sin recargar todo el documento. La técnica puede aplicarse a situaciones más complejas, como consultar productos, mostrar registros de estudiantes, validar formularios o recuperar datos de un sistema empresarial.

Aunque el ejemplo utiliza información ficticia, el mismo principio puede aplicarse a una aplicación real que disponga de un servidor y una API propia.

Conclusión

AJAX es una técnica fundamental en el desarrollo de aplicaciones web dinámicas, ya que permite intercambiar información entre el navegador y el servidor sin necesidad de recargar completamente una página. Su utilización contribuye a mejorar la interacción con los usuarios, reducir transferencias innecesarias de información y crear interfaces más fluidas.

A lo largo de esta investigación se explicó que AJAX no es un lenguaje de programación independiente, sino una combinación de tecnologías en la que JavaScript desempeña un papel importante. También se describió la relación entre el navegador, el servidor y los mecanismos de comunicación HTTP, así como el proceso mediante el cual una solicitud se envía, se procesa y produce una actualización en la página.

Entre sus aplicaciones se encuentran las búsquedas dinámicas, la validación de formularios, la carga de registros, la consulta de servicios web y la actualización de información en sistemas empresariales. Sin embargo, para aprovechar sus beneficios es necesario considerar los errores de conexión, la seguridad, la validación de los datos y la accesibilidad de la interfaz.

El ejemplo práctico permitió comprobar cómo una página puede solicitar información mediante Fetch API y presentar los resultados utilizando JavaScript. De esta forma, se demuestra que AJAX sigue siendo un concepto relevante, incluso cuando las aplicaciones modernas utilizan herramientas y bibliotecas más avanzadas.

En conclusión, comprender AJAX es importante para los estudiantes de Informática porque proporciona conocimientos esenciales sobre la comunicación entre aplicaciones web y servidores. Su dominio facilita la creación de sistemas interactivos, eficientes y adaptados a las necesidades actuales de los usuarios.

Referencias bibliográficas

Las siguientes fuentes técnicas se utilizaron para fundamentar las definiciones y los aspectos relacionados con AJAX y Fetch API.

MDN Web Docs. (s. f.-a). Ajax. Mozilla. https://developer.mozilla.org/en-US/docs/Glossary/AJAX

MDN Web Docs. (s. f.-b). Using the Fetch API. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

MDN Web Docs. (s. f.-c). XMLHttpRequest API. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest_API

JSONPlaceholder. (s. f.). JSONPlaceholder: Free fake REST API. https://jsonplaceholder.typicode.com/